

Practical 6: Traffic Sign Detection using CNNs

Objective:

To understand, implement, and analyze a Convolutional Neural Network (CNN) for traffic sign detection, and integrate it with XML parsing for label handling and data management.

Theory:

What is Traffic Sign Detection?

Traffic sign detection is a crucial component of advanced driver-assistance systems (ADAS) and autonomous vehicles. It involves identifying and classifying traffic signs in images or videos, providing essential information for safe and efficient navigation.

Why Use CNNs for Traffic Sign Detection?

Convolutional Neural Networks (CNNs) are particularly well-suited for image recognition tasks due to their ability to automatically learn hierarchical features from raw pixel data. Key advantages include:

- **Feature Extraction:** CNNs learn relevant features (edges, shapes, textures) directly from the images, eliminating the need for manual feature engineering.
- **Spatial Hierarchy:** CNNs capture spatial relationships between features at different scales through convolutional layers and pooling layers.
- **Robustness:** CNNs can be trained to be robust to variations in illumination, viewpoint, and partial occlusions.

CNN Model Architecture:

The CNN model used typically consists of the following layers:

- **Convolutional Layers (Conv2D):** These layers apply filters to the input image to extract feature maps. The `relu` (Rectified Linear Unit) activation function introduces non-linearity, enabling the network to learn complex patterns.
- **Max Pooling Layers (MaxPooling2D):** These layers reduce the spatial dimensions of the feature maps, making the network more robust to variations in the position of features. They also help to reduce computational complexity.
- **Flatten Layer:** This layer transforms the 2D feature maps into a 1D vector, preparing the output for the fully connected layers.
- **Dense (Fully Connected) Layers:** These layers perform classification based on the extracted features. A `relu` activation is often used in intermediate dense layers.
- **Dropout Layer:** This layer randomly sets a fraction of input units to 0 during training, preventing overfitting.

- **Output Layer:** This layer produces a probability distribution over the different traffic sign classes using the softmax activation function. The number of neurons in this layer corresponds to the number of traffic sign categories being detected.

Model Training:

- **Loss Function:** `categorical_crossentropy` is a common loss function used for multi-class classification problems. It measures the difference between the predicted probability distribution and the true class labels.
- **Optimizer:** `adam` is an adaptive optimization algorithm that adjusts the learning rate during training to efficiently minimize the loss function.

What is XML Parsing?

XML (Extensible Markup Language) parsing is the process of reading and extracting data from XML files. In this context, XML files often contain annotations for the traffic sign images, including bounding box coordinates and class labels.

Why Use XML Parsing for Traffic Sign Detection?

XML parsing is used to:

- **Read Annotations:** Extract the bounding box coordinates of traffic signs within the images from the XML annotation files.
- **Get Class Labels:** Determine the corresponding class label for each traffic sign instance.
- **Data Preparation:** Prepare the data for training and evaluation by associating the image regions (defined by the bounding boxes) with their corresponding class labels.

XML Parsing Library (`xml.etree.ElementTree`):

The `xml.etree.ElementTree` module in Python provides tools for parsing and manipulating XML documents.

Applications:

- **Autonomous Driving:** Enabling vehicles to perceive and understand traffic regulations.
- **Advanced Driver-Assistance Systems (ADAS):** Providing warnings and assistance to drivers based on detected traffic signs.
- **Traffic Monitoring:** Analyzing traffic sign compliance and identifying damaged or missing signs.
- **Mapping and Navigation:** Updating map databases with accurate traffic sign information.

```
In [1]: import os
import cv2
import numpy as np
import tensorflow as tf
import xml.etree.ElementTree as ET
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.utils import to_categorical
from sklearn.model_selection import train_test_split
```

```
In [2]: # Dataset paths
annotation_dir = "/kaggle/input/road-sign-detection/annotations"
image_dir = "/kaggle/input/road-sign-detection/images"

# Function to parse a single XML file
def parse_annotation(xml_file):
    tree = ET.parse(xml_file)
    root = tree.getroot()

    filename = root.find("filename").text
    objects = []

    for obj in root.findall("object"):
        label = obj.find("name").text
        bbox = obj.find("bndbox")

        xmin = int(bbox.find("xmin").text)
        ymin = int(bbox.find("ymin").text)
        xmax = int(bbox.find("xmax").text)
        ymax = int(bbox.find("ymax").text)

        objects.append((label, xmin, ymin, xmax, ymax))

    return filename, objects
```

```
In [3]: # Load images & labels
images = []
labels = []
target_size = (64, 64) # Resize all crops to this size

for xml_file in os.listdir(annotation_dir):
    if xml_file.endswith(".xml"):
        file_path = os.path.join(annotation_dir, xml_file)
        filename, objects = parse_annotation(file_path)

        img_path = os.path.join(image_dir, filename)
        if os.path.exists(img_path):
            img = cv2.imread(img_path)
            img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
            for label, xmin, ymin, xmax, ymax in objects:
                cropped = img[ymin:ymax, xmin:xmax]
                cropped = cv2.resize(cropped, target_size)
                cropped = cropped / 255.0 # Normalize
                images.append(cropped)
                labels.append(label)
```

```
images = np.array(images, dtype=np.float32)
labels = np.array(labels)
```

In [4]:

```
In [5]: # Encode labels
label_encoder = LabelEncoder()
labels_encoded = label_encoder.fit_transform(labels)
labels_categorical = to_categorical(labels_encoded)
```

```
In [6]: # Split into train & validation sets
X_train, X_val, y_train, y_val = train_test_split(
    images, labels_categorical, test_size=0.2, random_state=42 )

print("Train:", X_train.shape, y_train.shape)
print("Validation:", X_val.shape, y_val.shape)
print("Classes:", label_encoder.classes_)
```

```
Train: (995, 64, 64, 3) (995, 4)
Validation: (249, 64, 64, 3) (249, 4)
Classes: ['crosswalk' 'speedlimit' 'stop' 'trafficlight']
```

```
In [7]: # CNN model
model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(64, 64,3)),
    tf.keras.layers.MaxPooling2D(2,2),

    tf.keras.layers.Conv2D(64, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2,2),

    tf.keras.layers.Conv2D(128, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2,2),

    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dropout(0.5),

    tf.keras.layers.Dense(len(label_encoder.classes_), activation='softmax') ])

# Compile model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Par
conv2d (Conv2D)	(None, 62, 62, 32)	
max_pooling2d (MaxPooling2D)	(None, 31, 31, 32)	
conv2d_1 (Conv2D)	(None, 29, 29, 64)	18
max_pooling2d_1 (MaxPooling2D)	(None, 14, 14, 64)	
conv2d_2 (Conv2D)	(None, 12, 12, 128)	73
max_pooling2d_2 (MaxPooling2D)	(None, 6, 6, 128)	
flatten (Flatten)	(None, 4608)	
dense (Dense)	(None, 128)	589
dropout (Dropout)	(None, 128)	
dense_1 (Dense)	(None, 4)	

Total params: 683,716 (2.61 MB)

Trainable params: 683,716 (2.61 MB)

Non-trainable params: 0 (0.00 B)

```
In [8]: # Train
history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=15,
    batch_size=32 )
```

Epoch 1/15

32/32 ————— 7s 135ms/step - accuracy: 0.6353 - loss: 0.8832 -
val_accuracy: 0.8916 - val_loss: 0.2954

Epoch 14/15

32/32 ————— 4s 125ms/step - accuracy: 0.9908 - loss: 0.0273 -
val_accuracy: 0.9960 - val_loss: 0.0311

Epoch 15/15

32/32 ————— 4s 124ms/step - accuracy: 1.0000 - loss: 0.0067 -
val_accuracy: 0.9920 - val_loss: 0.0286

```
In [9]: # Save model & label encoder
model.save("traffic_sign_cnn.h5")

import pickle
with open("label_encoder.pkl", "wb") as f:
    pickle.dump(label_encoder, f)
```